

Machine Learning 1

Lecture 9

Mixture of Experts

Alejandro Monroy

Outline

Latent Variable Models	3
Mixture of Experts	9
Constrained Optimization	13
Training GMMs	18
K-Means	29
Applications of Mixture Models	36

Lecture 9

Mixture of Experts

Alejandro Monroy

ML1 so far

Probabilistic modelling & Estimation

We can fit the parameters of a probabilistic model $p(x \mid \theta)$ to a dataset $\mathcal{D}$ by

Maximum Likelihood Estimation

$$\hat{\theta}_{ML} = \operatorname{argmax}_{\theta} \log p(\mathcal{D} \mid \theta)$$

Maximum a Posteriori Estimation

$$\hat{\theta}_{MAP} = \operatorname{argmax}_{\theta} p(\theta \mid \mathcal{D})$$

Supervised learning

Given labeled data $\mathcal{D} = \{\mathbf{X}, \mathbf{T}\}$, we want to learn a model $f(\mathbf{x}) \simeq \mathbf{t}$.

The target can be continuous (**regression**) or discrete (**classification**)

Unsupervised learning

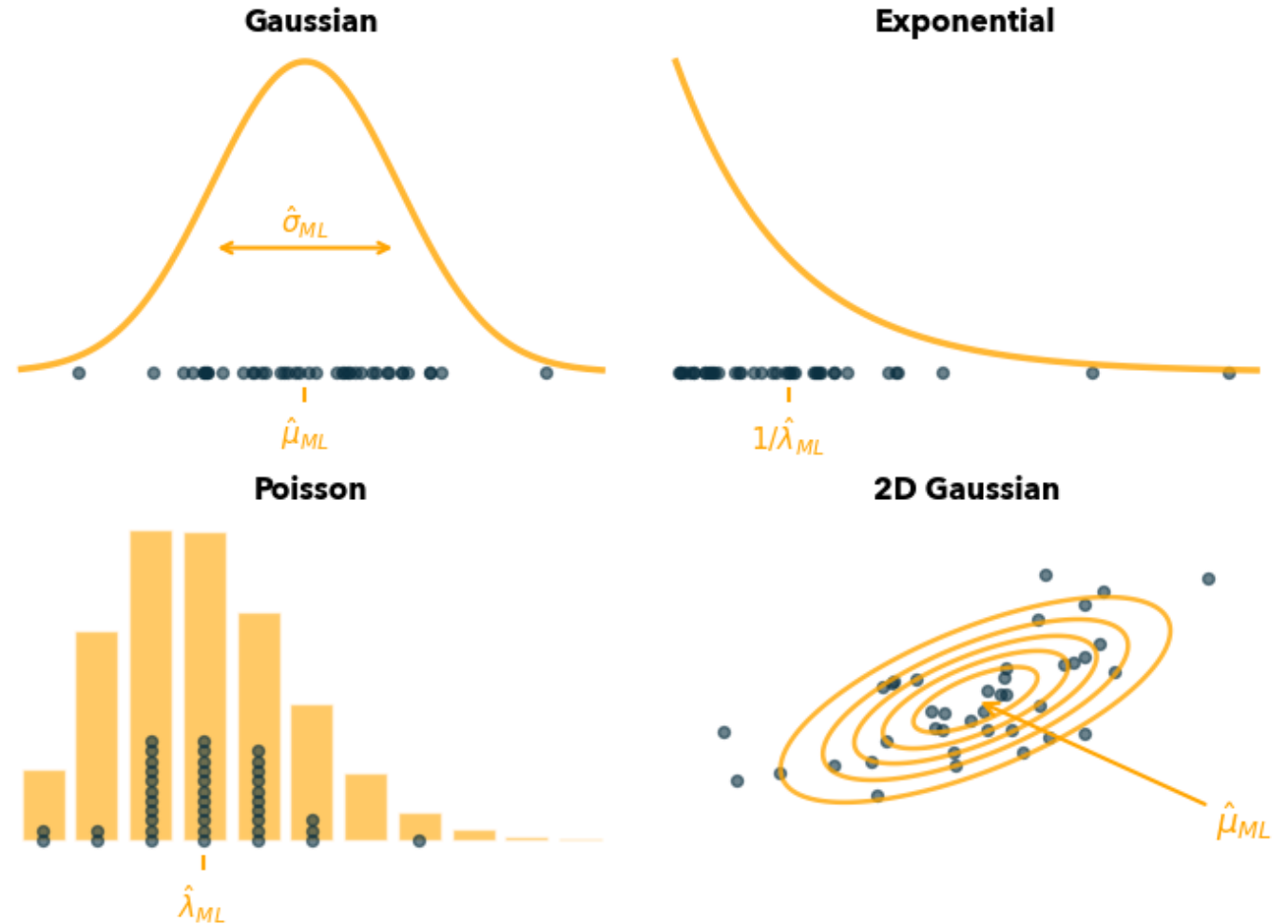
Given unlabeled data $\mathcal{D} = \{\mathbf{X}\}$, we want to learn the structure of the data:

$p(\mathbf{x}), p(\mathbf{z} \mid \mathbf{x}), p(\mathbf{x} \mid \mathbf{z})$,
which we can then use for tasks such as
clustering or **dimensionality reduction**

Latent variable models

Probabilistic modelling so far

In week 1 we learned how to fit the parameters of a probability distribution to a set of samples



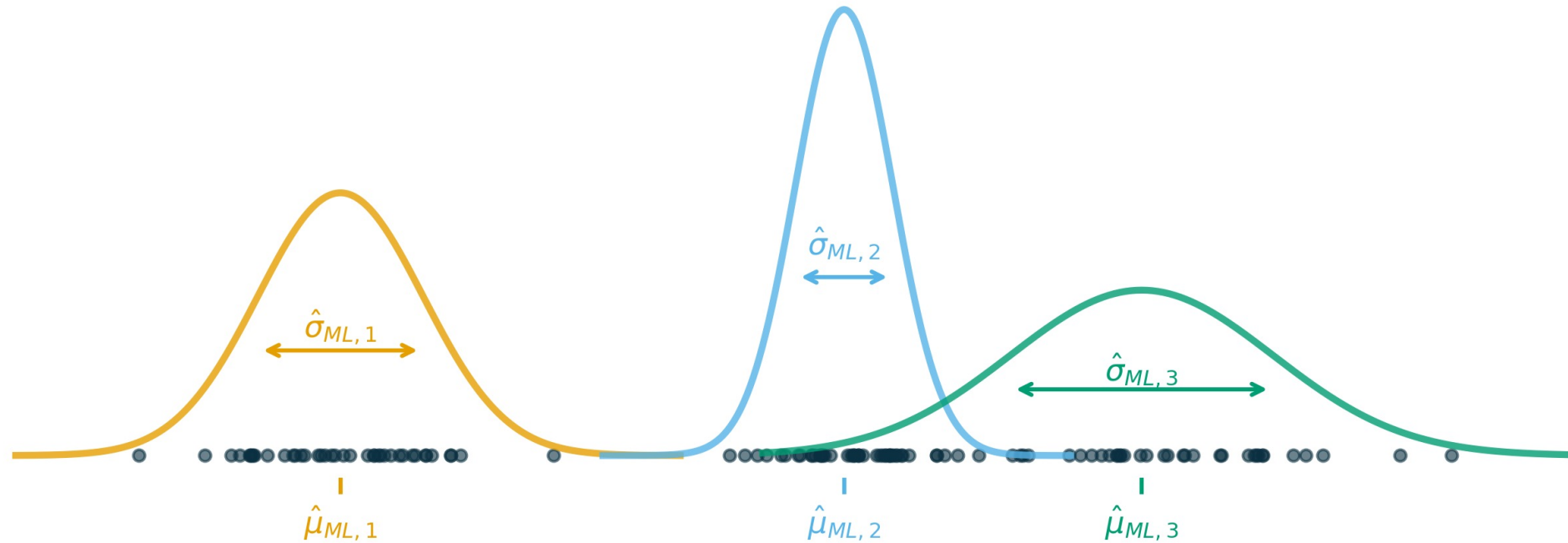
More complex models

Inductive bias: our data comes from different populations



More complex models

Inductive bias: our data comes from different populations

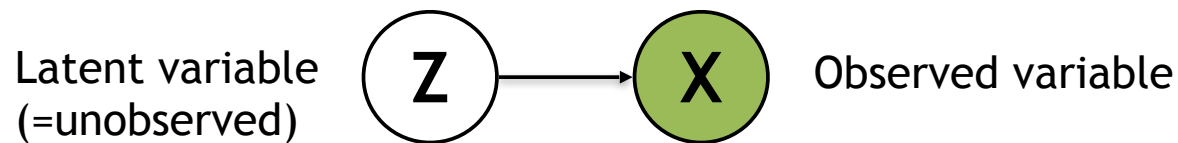


Latent variable models

The density of a **latent variable model** can be written as

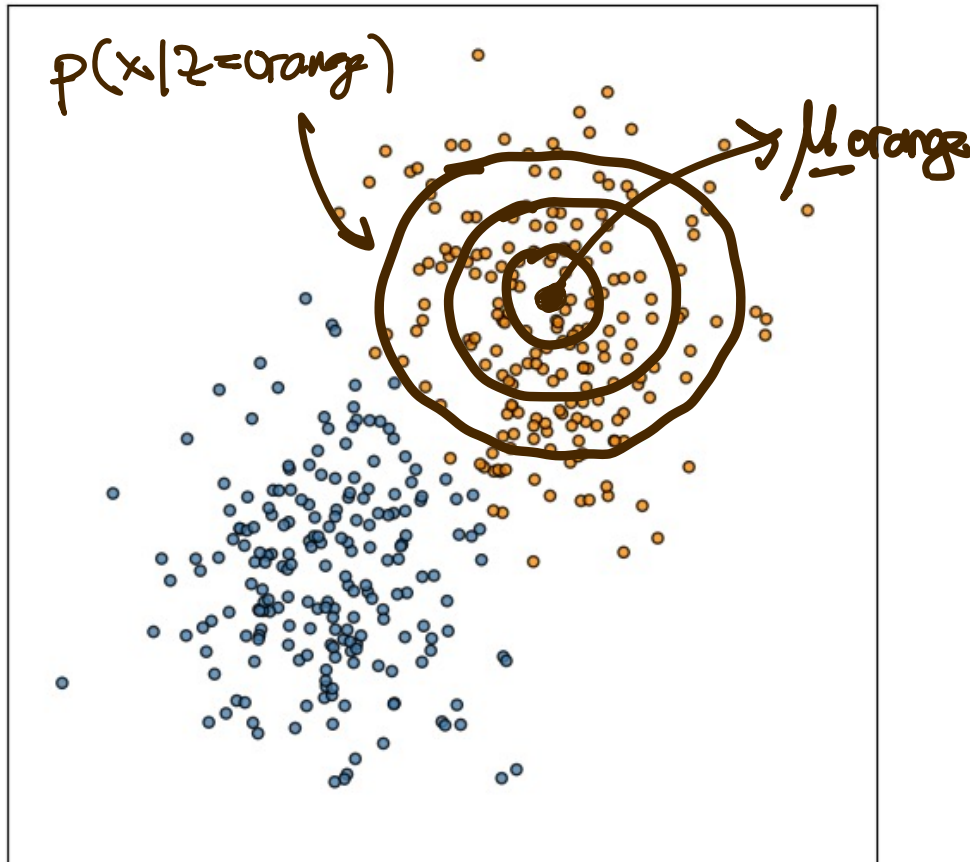
$$p(x) = \int_{\mathbf{z}} p(\mathbf{z})p(x | \mathbf{z})d\mathbf{z} \quad \Bigg| \quad p(x) = \sum_{\mathbf{z}} p(\mathbf{z})p(x | \mathbf{z})$$

- $\mathbf{z}$ is a **latent variable** with prior $p(\mathbf{z})$
- The (hidden) latent variable $\mathbf{z}$ influences the distribution of the data ($p(x | \mathbf{z})$)

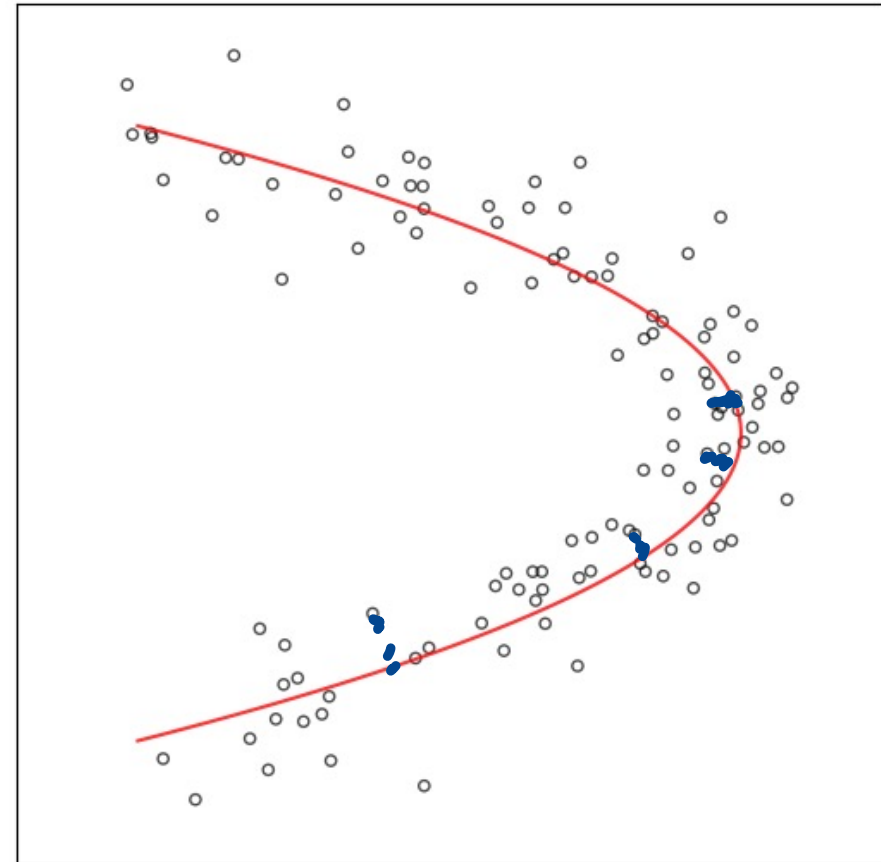


Latent variable models – Examples

Clustering
(Discrete latent variable)



Dimensionality reduction
(Continuous latent variable)



Mixture of Experts

Mixture models

The density of a **mixture model** can be written as

$$p(\mathbf{x}) = \sum_z p(z)p(\mathbf{x} | z)$$

- z is a discrete **latent variable** that encodes the class/cluster
 - We model z as a categorical distribution, 1-hot encoded as a vector of K binary variables

$$p(z_k = 1) = \pi_k, \quad \pi_k \in [0,1], \quad \sum_{k=1}^K \pi_k = 1$$

- The **class conditional probabilities** $p(x | z)$ are different for every class.
 - For example, they can be modelled with the same type of distribution with different hyperparameters depending on the class, or even different types of distributions.

Gaussian mixture models (GMMs)

We will focus on a specific example, where all class conditional experts are Gaussian:

$$p(\mathbf{x} \mid \boldsymbol{\pi}, \{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\}) = \sum_k \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}_k|^{1/2}} e^{-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)}.$$

Note that here we are making the following modelling choices:

- We model $\mathbf{z}$ as a categorical distribution, 1-hot encoded as a vector of K binary variables

$$p(z_k = 1) = \pi_k, \quad \pi_k \in [0,1], \quad \sum_{k=1}^K \pi_k = 1$$

- All class conditional distributions are (multivariate) Gaussians with different mean $\boldsymbol{\mu}_k$ and covariance matrix $\boldsymbol{\Sigma}_k$ per class.

Gaussian mixture models - Optimization

$\hookrightarrow \pi_k \}_{k=1}^K$ $K \leftarrow \# \text{ of classes}$

Goal: Given a mixture model $p(x \mid \{\pi_k\}, \{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\})$, we want to compute optimal values for the parameters $\{\pi_k\}, \{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\}$ based on observed data $\mathcal{D} = \{x_1, \dots, x_N\}$.

We know how to do this $\rightarrow$ **Maximum Likelihood Estimation**

- For the Gaussian parameters, we solve

$$\{\hat{\boldsymbol{\mu}}_k\}, \{\hat{\boldsymbol{\Sigma}}_k\} = \operatorname{argmax}_{\{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\}} \log p(\mathcal{D} \mid \{\pi_k\}, \{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\})$$

- For the class probabilities, the solution needs to satisfy a **constraint!**

$$\{\pi_k\} = \operatorname{argmax}_{\{\pi_k\}} \log p(\mathcal{D} \mid \{\pi_k\}, \{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\}) \text{ subject to } \sum_{k=1}^K \pi_k = 1$$

Constrained Optimization

Unconstrained vs constrained optimization

So far, we have solved unconstrained optimization problems of the form

$$\max_{\mathbf{x}} f(\mathbf{x})$$

However, when the solutions for ~~$\mathbf{x}$~~ ^{$\mathbf{x}$} need to satisfy some constraint, we have to solve a constrained optimization problem of the form

$$\max_{\mathbf{x}} f(\mathbf{x}) \text{ subject to } g(\mathbf{x}) = c, c \in \mathbb{R}$$

Unconstrained optimization – Lagrange Multipliers

The **method of Lagrange multipliers** allows us to solve problems of the form

$$\max_{\mathbf{x}} f(\mathbf{x}) \text{ subject to } g(\underline{\mathbf{x}}) = c, c \in \mathbb{R}$$

by:

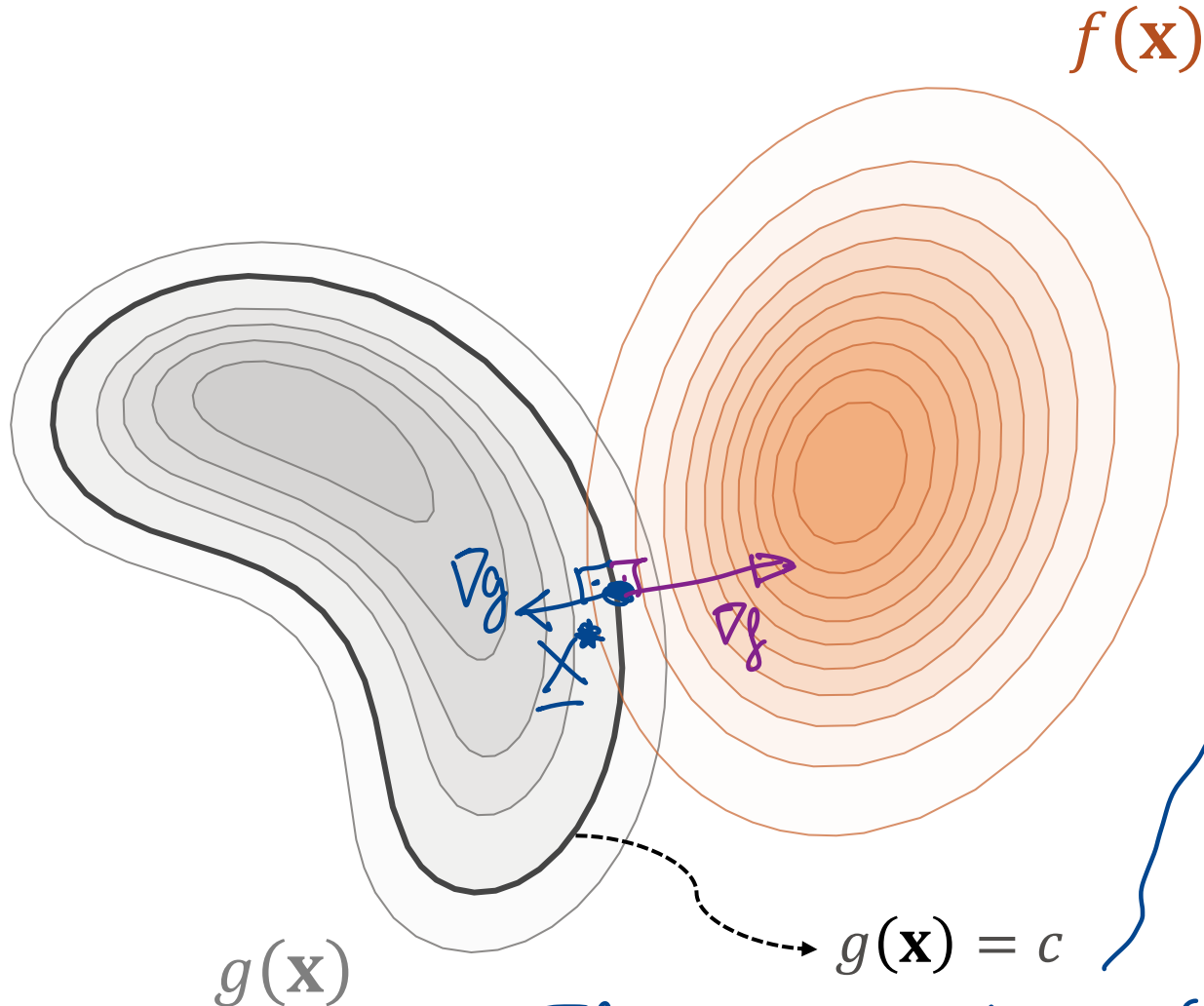
1. Defining the **Lagrangian**:

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda(g(\mathbf{x}) - c)$$

2. Finding the stationary point for both ~~$\mathbf{x}$~~ and λ :

$$\frac{\partial}{\partial \mathbf{x}} L(\mathbf{x}, \lambda) = 0 \text{ and } \frac{\partial}{\partial \lambda} L(\underline{\mathbf{x}}, \lambda) = 0$$

Unconstrained optimization – Lagrange Multipliers



Constrained optimization problem:

$$\max_{\mathbf{x}} f(\mathbf{x}) \text{ subject to } g(\mathbf{x}) = c$$

Method of Lagrange multipliers:

1. Defining the **Lagrangian**:

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda(g(\mathbf{x}) - c)$$

2. Finding the stationary point:

$$\frac{\partial}{\partial \mathbf{x}} L(\mathbf{x}, \lambda) = 0 \text{ and } \frac{\partial}{\partial \lambda} L(\mathbf{x}, \lambda) = 0$$

$$\nabla f + \lambda \nabla g = 0, \quad \lambda \in \mathbb{R} \text{ (in this case, } \lambda \neq 0 \text{)}$$

$$g(\mathbf{x}) = c$$

Constrained optimization - Toy example

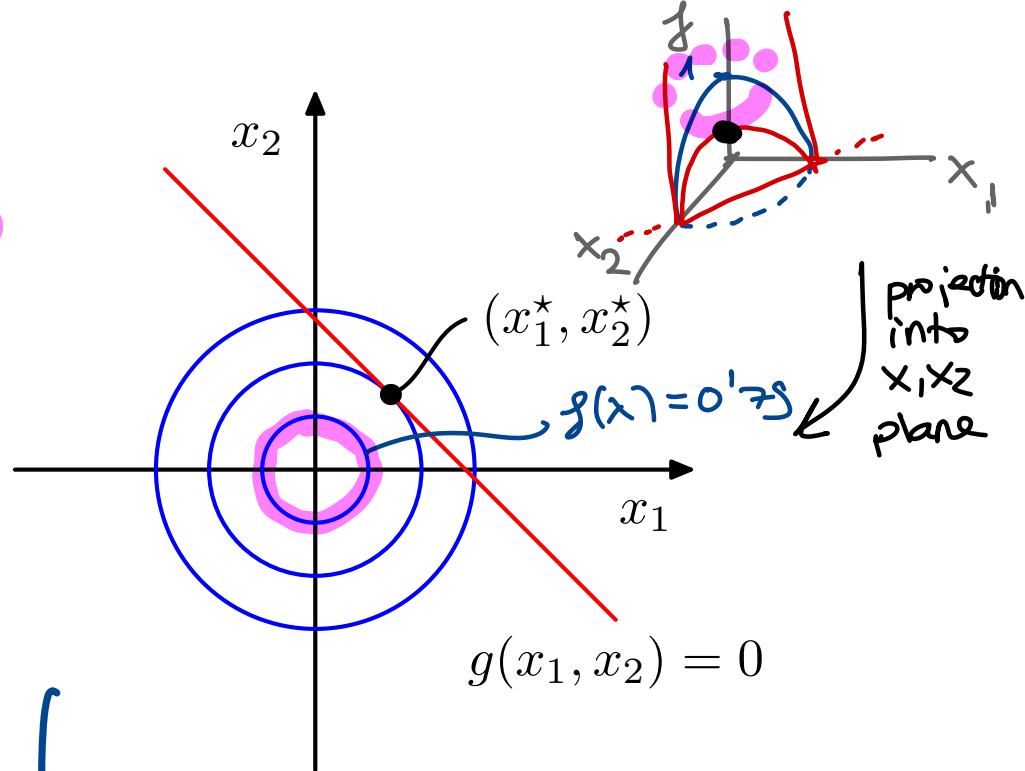
Figure E.2 A simple example of the use of Lagrange multipliers in which the aim is to maximize $f(x_1, x_2) = 1 - x_1^2 - x_2^2$ subject to the constraint $g(x_1, x_2) = 0$ where $g(x_1, x_2) = x_1 + x_2 - 1$. The circles show contours of the function $f(x_1, x_2)$, and the diagonal line shows the constraint surface $g(x_1, x_2) = 0$.

$$L = 1 - x_1^2 - x_2^2 + \lambda(x_1 + x_2 - 1)$$

$$\frac{\partial L}{\partial x_1} = -2x_1 + \lambda \stackrel{!}{=} 0 \Rightarrow x_1 = \frac{\lambda}{2}$$

$$\frac{\partial L}{\partial x_2} = -2x_2 + \lambda \stackrel{!}{=} 0 \Rightarrow x_2 = \frac{\lambda}{2}$$

$$\frac{\partial L}{\partial \lambda} = x_1 + x_2 - 1 \stackrel{!}{=} 0 \Rightarrow \lambda = 1$$



$$\Rightarrow (x_1^*, x_2^*) = \left(\frac{1}{2}, \frac{1}{2}\right)$$

Training a Gaussian Mixture Model

Gaussian mixture models - Optimization

omitted from now on

Goal: Given a mixture model $p(x \mid \{\pi_k\}, \{\mu_k\}, \{\Sigma_k\})$, we want to compute optimal values for the parameters $\{\pi_k\}, \{\mu_k\}, \{\Sigma_k\}$ based on observed data $\mathcal{D} = \{x_1, \dots, x_N\}$.

We know how to do this → **Maximum Likelihood Estimation**

- For the Gaussian parameters, we solve

$$\{\hat{\mu}_k\}, \{\hat{\Sigma}_k\} = \operatorname{argmax}_{\{\mu_k\}, \{\Sigma_k\}} \log p(\mathcal{D} \mid \{\pi_k\}, \{\mu_k\}, \{\Sigma_k\})$$

- For the class probabilities, the solution needs to satisfy a **constraint!**

$$\{\pi_k\} = \operatorname{argmax}_{\{\pi_k\}} \log p(\mathcal{D} \mid \{\pi_k\}, \{\mu_k\}, \{\Sigma_k\}) \text{ subject to } \sum_{k=1}^K \pi_k = 1$$

Optimizing GMMs - Responsibilities

Given a GMM $p(\mathbf{x}) = \sum_k \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$, how can we compute the posterior class probabilities?

$$\begin{aligned} p(z_k = 1 | \mathbf{x}) &= \frac{p(z_k = 1) p(\mathbf{x} | z_k = 1)}{p(\mathbf{x})} \\ &= \frac{p(z_k = 1) p(\mathbf{x} | z_k = 1)}{\sum_j p(z_j = 1) p(\mathbf{x} | z_j = 1)} \\ &= \frac{\pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} = \gamma(z_k) \end{aligned}$$

$$\sum_{k=1}^K \gamma(z_k) = 1$$

← responsibility
of expert k
for datapoint
 $\mathbf{x}$

Optimizing GMMs – Log likelihood

Given a GMM $p(\mathbf{x}) = \sum_k \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$, the log-likelihood of a dataset $\mathcal{D}$ is

$$\begin{aligned}\ln p(\mathcal{D}) &= \ln \prod_{n=1}^N p(\mathbf{x}_n) && (\text{i. i. d.}) \\ &= \sum_{n=1}^N \ln p(\mathbf{x}_n) \\ &= \sum_{n=1}^N \ln \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\end{aligned}$$

Optimizing GMMs - Gaussian parameters

Optimal value for μ_k :

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}_k|^{1/2}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x}-\boldsymbol{\mu}_k)}.$$

$$\frac{\partial}{\partial \underline{\mu}_k} \sum_{n=1}^N \log p(\mathbf{x}_n) = \sum_{n=1}^N \frac{1}{p(\mathbf{x}_n)} \frac{\partial}{\partial \underline{\mu}_k} p(\mathbf{x}_n)$$

(chain rule)

$$= \sum_{n=1}^N \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \underline{\mu}_k, \underline{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n | \underline{\mu}_j, \underline{\Sigma}_j)} (\mathbf{x}_n - \underline{\mu}_k)^T \underline{\Sigma}_k^{-1} \stackrel{!}{=} 0 \text{ and solve for } \underline{\mu}_k$$

$\Rightarrow \delta(z_{nk})$
(fixed)

$\uparrow$
we only optimize w.r.t this $\underline{\mu}_k$

$\downarrow$
no analytical solution

Optimizing GMMs – Gaussian parameters

Optimal value for $\boldsymbol{\mu}_k$:

$$\mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi)^{d/2} |\boldsymbol{\Sigma}_k|^{1/2}} e^{-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)}.$$

$$\begin{aligned} \frac{\partial}{\partial \boldsymbol{\mu}_k} \sum_{n=1}^N \log p(\mathbf{x}_n) &= \sum_{n=1}^N \frac{1}{p(\mathbf{x}_n)} \frac{\partial}{\partial \boldsymbol{\mu}_k} p(\mathbf{x}_n) \\ &= \sum_{n=1}^N \frac{\pi_k \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} (\mathbf{x}_n - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} \\ &= \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} \stackrel{!}{=} 0 \\ \Rightarrow \boldsymbol{\mu}_k &= \frac{\sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n}{\sum_{n=1}^N \gamma(z_{nk})} \end{aligned}$$

Optimizing GMMs – Gaussian parameters

Applying a similar procedure with Σ_k , we obtain

$$\Sigma_k = \frac{1}{\sum_{n=1}^N \gamma(z_{nk})} \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top$$

Optimizing GMMs – Class probabilities

Finally, let's optimize for π_k :

Lagrangian: $L(\{\pi_k\}, \lambda) = \overbrace{\sum_{n=1}^N \log p(\mathbf{x}_n)}^{\text{objective fn}} + \lambda \overbrace{(\sum_{j=1}^K \pi_j - 1)}^{\text{constraint}}$

$$\frac{\partial}{\partial \pi_k} L(\{\pi_k\}, \lambda) = \sum_{n=1}^N \frac{\mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} + \lambda \stackrel{!}{=} 0 \quad \Rightarrow \quad \pi_k = -\frac{1}{\lambda} \sum_{n=1}^N \gamma(z_{nk}) \quad (*)$$

$$\frac{\partial}{\partial \lambda} L(\{\pi_k\}, \lambda) = \sum_{j=1}^K \pi_j - 1 = -\frac{1}{\lambda} \sum_{j=1}^K \sum_{n=1}^N \gamma(z_{nj}) - 1 \stackrel{!}{=} 0 \quad ; \quad -\frac{1}{\lambda} \underbrace{\sum_n \sum_j \gamma(z_{nj})}_N - 1 \stackrel{!}{=} 0$$

$$\Rightarrow \lambda = -N \quad (*) \quad \Rightarrow \quad \pi_k = \frac{1}{N} \sum_{n=1}^N \gamma(z_{nk})$$

Optimizing GMMs – Expectation-Maximization algorithm

Initialize $\{\pi_k\}, \{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\}$ (e.g. randomly)

Repeat until convergence:

Expectation step (E-step): Update the posterior

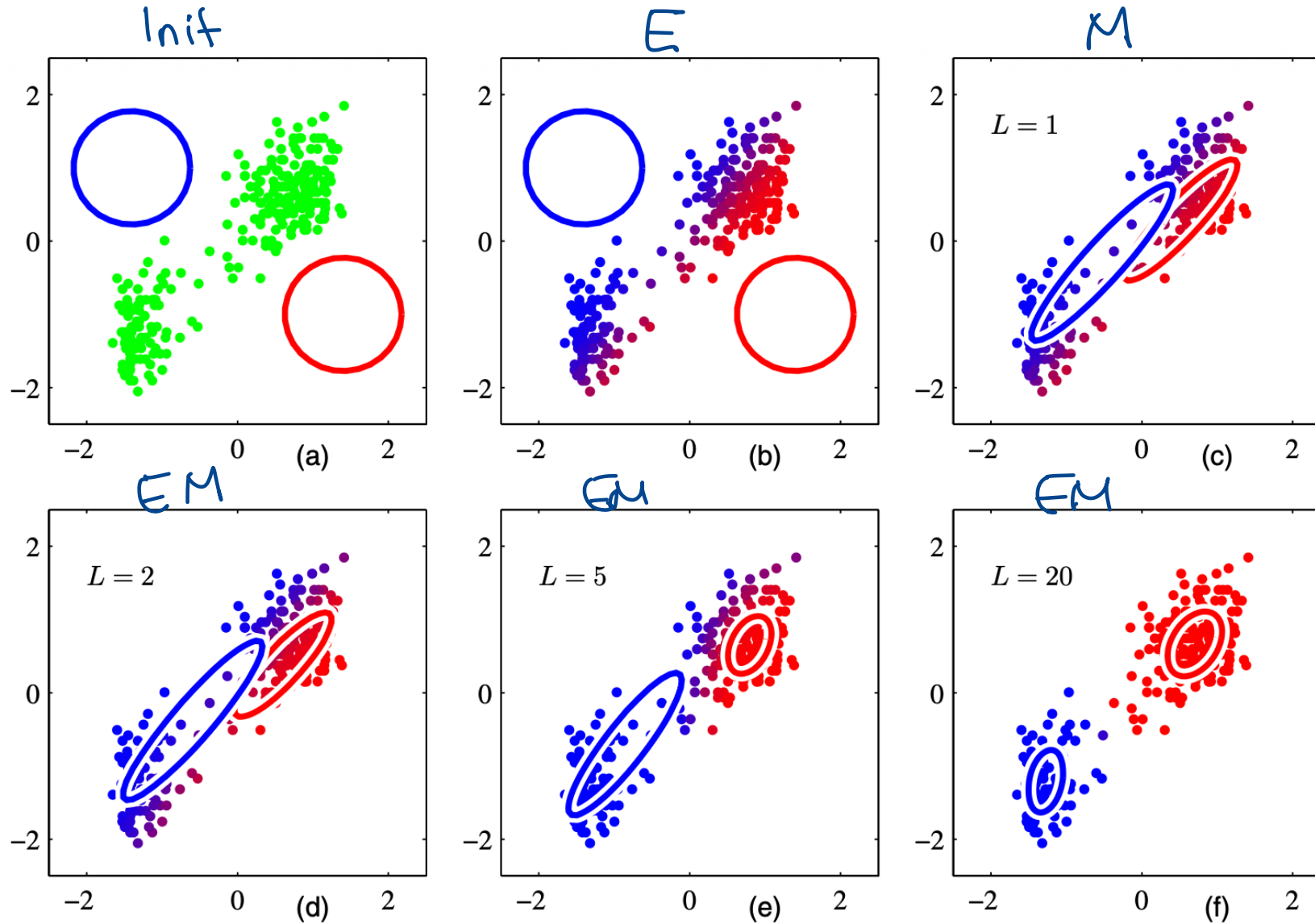
$$\gamma(z_{nk}) \leftarrow \frac{\pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

Maximization step (M-step): Update the parameters

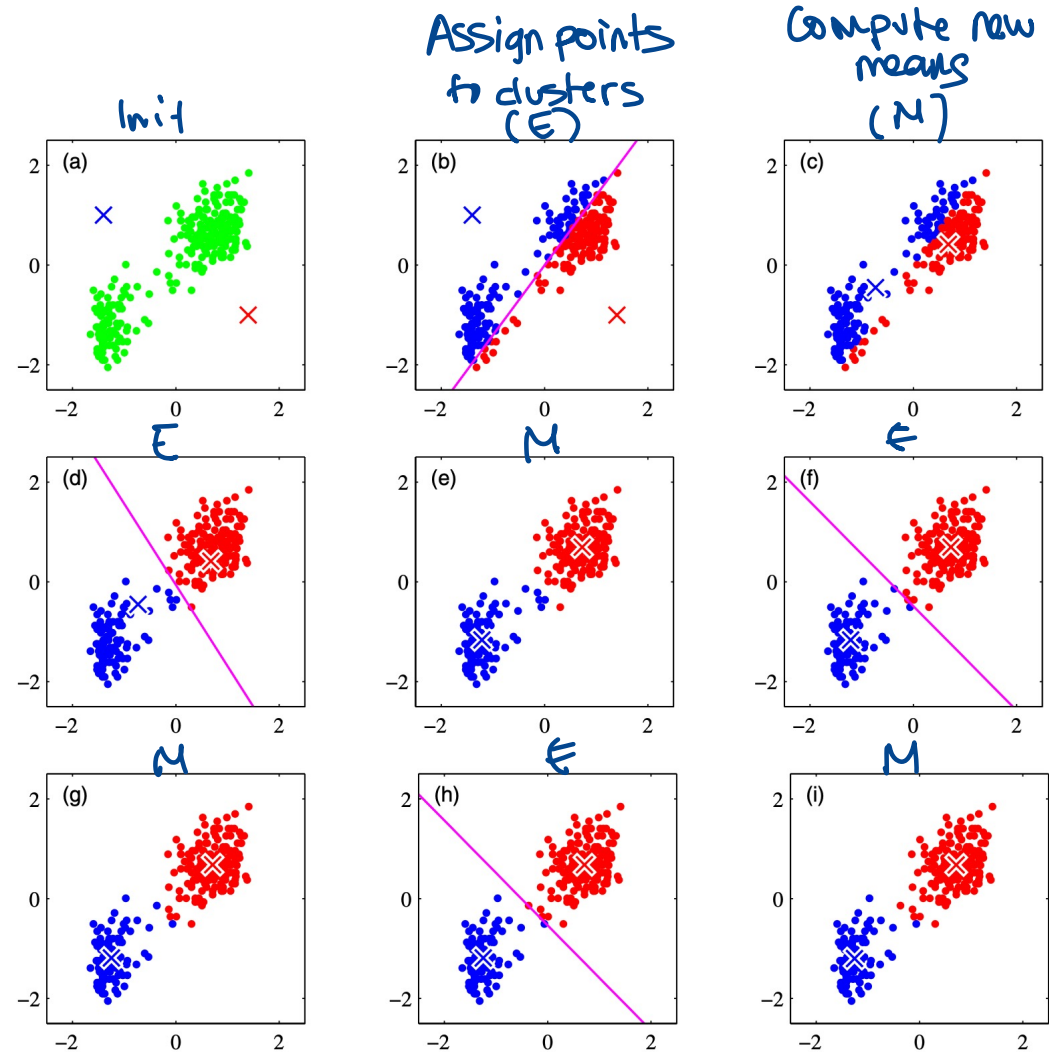
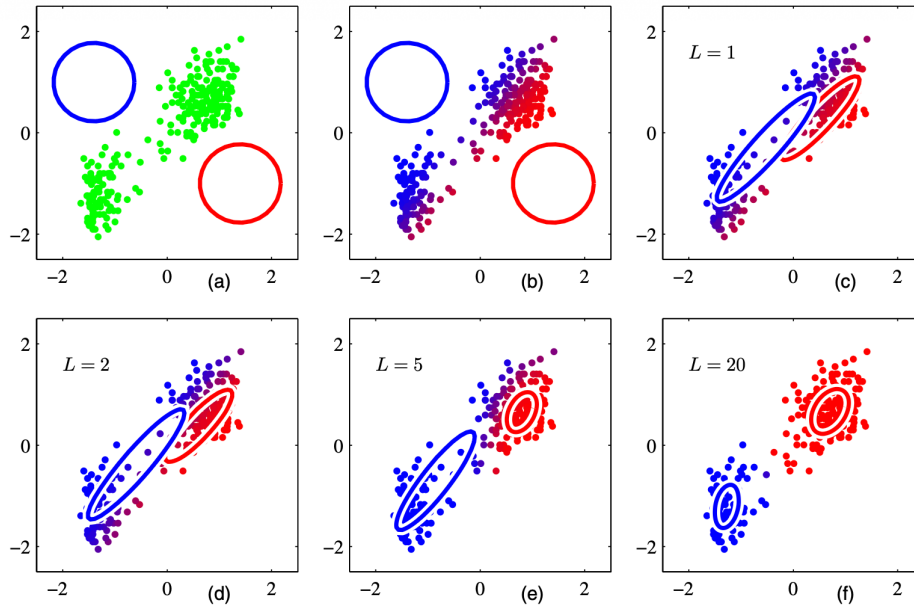
$$\boldsymbol{\mu}_k \leftarrow \frac{\sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n}{\sum_{n=1}^N \gamma(z_{nk})} \qquad \pi_k \leftarrow \frac{1}{N} \sum_{n=1}^N \gamma(z_{nk})$$

$$\boldsymbol{\Sigma}_k \leftarrow \frac{1}{\sum_{n=1}^N \gamma(z_{nk})} \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top$$

Optimizing GMMs – Expectation-Maximization algorithm



Hmm... Doesn't this seem familiar?



K-means

K-means algorithm

Algorithmic
view

K-means

$$\text{Loss: } J = \sum_{n=1}^N \sum_{k=1}^K z_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2$$

Initialize $\{\boldsymbol{\mu}_k\}$

Repeat until convergence:

Expectation step (E-step):

$$z_{nk} \leftarrow \begin{cases} 1 & \text{if } k = \arg \min_j \|\mathbf{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0 & \text{otherwise} \end{cases} \quad \left(\text{Hard cluster assignments} \right)$$

Maximization step (M-step):

$$\boldsymbol{\mu}_k \leftarrow \frac{\sum_{n=1}^N z_{nk} \mathbf{x}_n}{\sum_{n=1}^N z_{nk}} = \text{avg of the points in class } k$$

Mixture of Gaussians

Probabilistic
view

$$\text{Loss: } L = - \sum_{n=1}^N \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$$

Initialize $\{\pi_k\}, \{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\}$

Repeat until convergence:

Expectation step (E-step):

$$\gamma(z_{nk}) \leftarrow \frac{\pi_k \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_n | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

Maximization step (M-step):

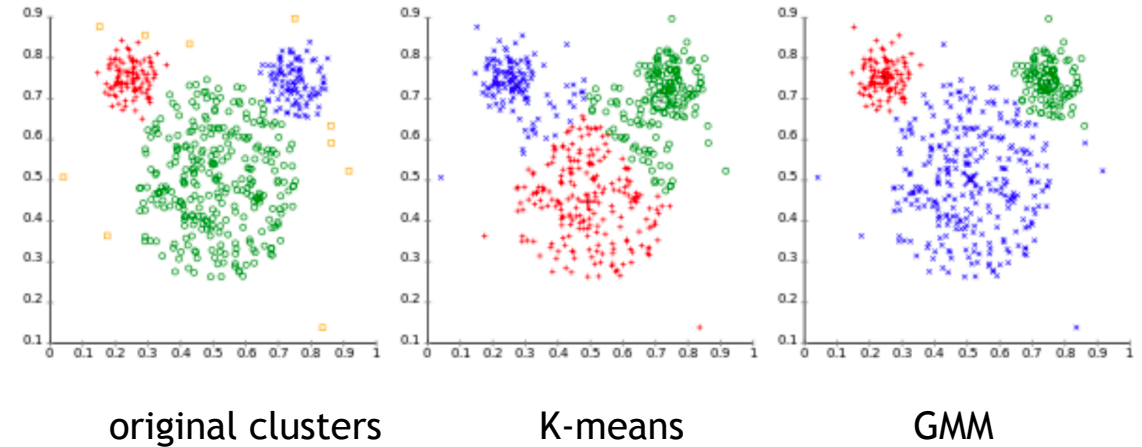
$$\boldsymbol{\mu}_k \leftarrow \frac{\sum_{n=1}^N \gamma(z_{nk}) \mathbf{x}_n}{\sum_{n=1}^N \gamma(z_{nk})} \quad \pi_k \leftarrow \frac{1}{N} \sum_{n=1}^N \gamma(z_{nk})$$

$$\boldsymbol{\Sigma}_k \leftarrow \frac{1}{\sum_{n=1}^N \gamma(z_{nk})} \sum_{n=1}^N \gamma(z_{nk}) (\mathbf{x}_n - \boldsymbol{\mu}_k)(\mathbf{x}_n - \boldsymbol{\mu}_k)^\top$$

K-means vs GMMs

	K-means	GMMs
Number of clusters	Fixed	
Class assignments	Hard assignments	Soft assignments
Cluster covariances	All equal and spherical	Different per cluster
Efficiency	Faster	Slower
Convergence	Local convergence, not guaranteed to find the global optimal	

(non-convex loss)

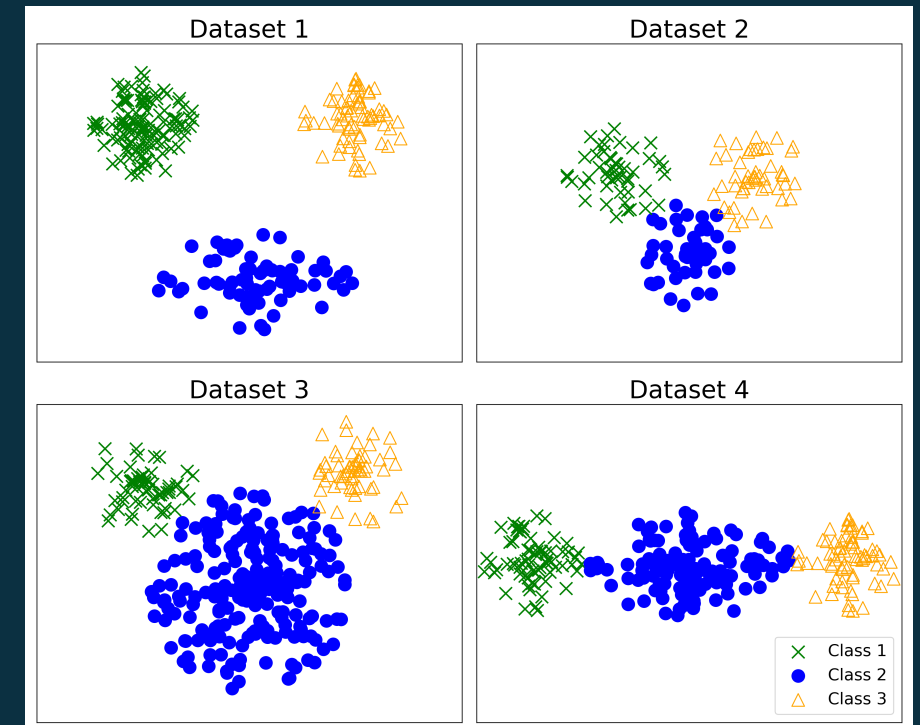


Quiz time!

For which of the datasets in the figure can the corresponding cluster assignments be the result of applying K-Means to the unlabeled data?



- a) Dataset 1
- b) Dataset 2
- c) Dataset 3
- d) Dataset 4

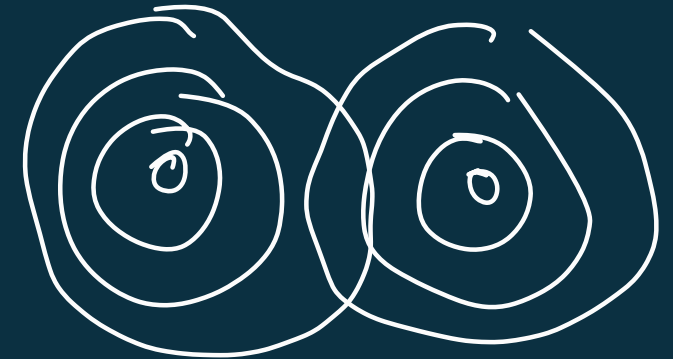


Quiz time!

For clustering purposes, under what constraint(s) does a Gaussian mixture model become equivalent to K-means?

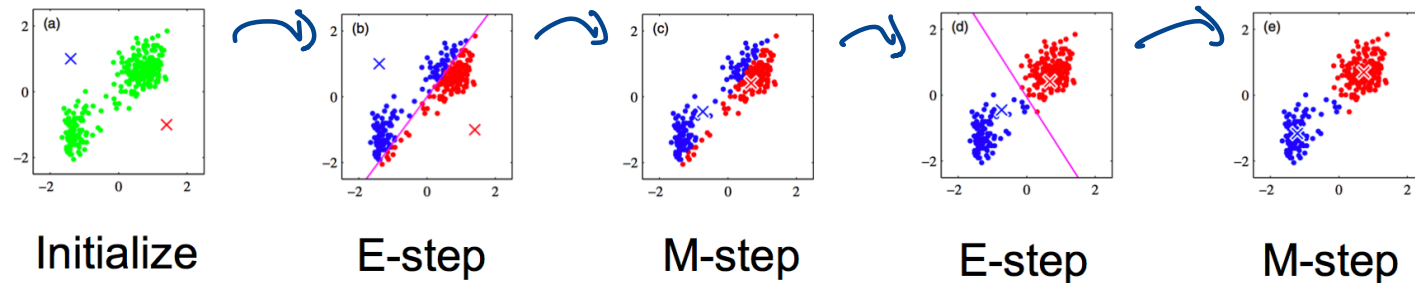
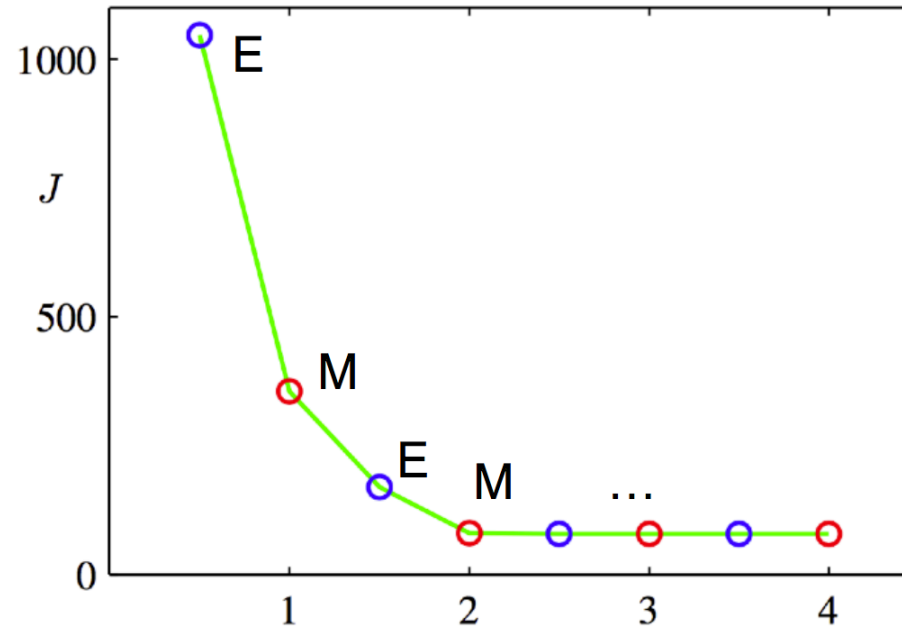


- a) $\pi_1 = \pi_2 = \dots = \pi_K$ (All components have equal mixing weights)
- b) $\Sigma_1 = \Sigma_2 = \dots = \Sigma_K$ (All components have equal covariance matrices)
- c) $\Sigma_1 = \Sigma_2 = \dots = \Sigma_K = \sigma^2 \mathbf{I}$ (All components have equal and isotropic/spherical covariance matrices)
- d) $\pi_1 = \pi_2 = \dots = \pi_K$ and $\Sigma_1 = \Sigma_2 = \dots = \Sigma_K$
- ☒ e) $\pi_1 = \pi_2 = \dots = \pi_K$ and $\Sigma_1 = \Sigma_2 = \dots = \Sigma_K = \sigma^2 \mathbf{I}$



Convergence of EM for GMMs and K-means

- After every E-step or M-step, the loss is guaranteed to decrease or remain the same (not increase)
- The algorithm converges, but it might converge to a local minimum, not the global one!



Mixture Models – Applications

Applications of mixture models

Once trained (i.e. both latent and class conditional distributions are known), we can use a mixture model for:

- Clustering a training datapoint $\mathbf{x}_n$:

1. Compute responsibilities $\{\gamma(z_{nk})\}_{k=1}^K$
2. The most likely cluster is given by $k = \arg \max_{j \in \{1, 2, \dots, K\}} \gamma(z_{nj})$

- Classifying new data

- Same procedure as for assigning clusters

- Generating a new datapoint $\mathbf{x}_0$:

1. Sample $z_0 \sim p(z)$
2. Sample $\mathbf{x}_0 \sim p(\mathbf{x}|z_0)$



That was it!

HW-based exam 1 results are out

– Discussions open until next Monday

HW-based exam 2 next Thursday

Lab 4 out tomorrow – Deadline next Monday

Next Lecture: Dimensionality Reduction

Please complete this
feedback form 🙏

